

They solve the same problem — run many Move transactions at once without breaking a serial story — from opposite ends. **Aptos discovers conflicts after the fact inside a totally ordered block.** **Sui encodes conflicts in the data model so many txs never enter that block at all.**

## One-line contrast

|                             | Aptos + Block-STM                        | Sui                                                    |
|-----------------------------|------------------------------------------|--------------------------------------------------------|
| When is independence known? | During / after execution                 | Before execution (object list + ownership)             |
| What is ordered?            | Every user tx, by BFT                    | Only txs that touch <b>shared</b> objects              |
| Conflict tool               | Optimistic STM + re-execute              | Causal / versioned objects; owned path skips consensus |
| Developer writes            | Ordinary <code>borrow_global</code> Move | Object IDs in, ownership as types                      |

## Architecture

|                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| text                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <div><div>APTOS</div><div>mempool → BFT orders a block <math>T_0 &lt; T_1 &lt; \dots T_n</math><br/>↓<br/>Block-STM workers<br/>MVMemory + validate + abort/re-exec<br/>↓<br/>commit write sets (per-tx versions)</div></div> <div><div>SUI</div><div>tx names objects + owners<br/>├ only owned / immutable → certify (fast path) → execute<br/>└ any shared object → Mysticeti order versions → execute in causal order</div></div> |

Aptos: consensus first, parallel execution second.

Sui: classify first; consensus is optional.

## State model (this is the real difference)

**Aptos** is still Diem-shaped: an address holds resources (and modules). Anyone who can name an address can `borrow_global` it. Aptos Objects (AIP-10) group resources at an object address, but the VM still uses global storage. Parallelism cannot be complete until the interpreter has touched keys.

**Sui** deleted global storage. Every asset is an object: `id`, `version`, `owner`. A tx must pass the objects it will use. Ownership classes:

- **Address-owned** — one writer; fast path
- **Immutable** — many readers, no version bump
- **Shared** — many writers; Mysticeti assigns versions; mutating the same shared object serializes

Reads of a shared object at version  $v$  do not conflict with the writer that produces  $v + 1$ . Only two **mutating** accesses of the same shared object form an execution edge.

That is why Sui looks like deterministic PCC and Aptos looks like OCC. The VM APIs forced the schedulers.

## How each runs a block of work

### Aptos Block-STM

1. Block is already totally ordered.
2. Every tx starts in  $E$  (execute).  $V$  (validate) is empty.
3. Workers run Move against MVMemory ( `read` = last writer with index  $< me$  ).
4. Validate read sets in order-priority.
5. Fail  $\rightarrow$  ESTIMATE, bump incarnation, pull `validation_idx` backward, re-execute.
6. Done when cursors pass  $n$  and no in-flight work.
7. Commit is equivalent to serial  $T_0 \dots T_{n-1}$ .

Cost when you guess wrong: extra VM runs + suffix re-validation. Benefit: no access lists; composable `borrow_global`.

### Sui execution

1. Client lists object refs (id + version).
2. If all owned by the sender (or immutable): Byzantine consistent broadcast / Mysticeti-FPC; execute as soon as certificates exist. No global slot vs every other player.
3. If any object is shared: consensus picks a version for that object; txs that write it form a queue on that object's version chain. Unrelated objects still run in parallel.
4. No speculative abort on the owned path. Shared-object execution is scheduled from the declared graph, not from a post-hoc read set.

Cost when you guess wrong: you cannot “discover” an extra object mid-tx; it had to be an input. Benefit: no wasted incarnations on the common path; owned transfers do not wait on a DEX.

## Latency and contention

| Workload                                                        | Block-STM                                                                             | Sui                                                                             |
|-----------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| A transfers NFT, B transfers different NFT                      | Parallel inside one block; still wait for block finality                              | Two fast-path certs; no shared queue                                            |
| 10k users hit one AMM pool                                      | Many aborts on pool resource; STM serializes the hot key                              | All txs name the pool; consensus orders pool versions; other objects unaffected |
| Read-only of a shared catalog                                   | Still a read in the STM read set (can abort if someone wrote it earlier in the block) | Shared read at pinned version; no conflict with later writer                    |
| Nested “bag of items”                                           | Dynamic <code>borrow_global</code> / table keys — STM infers                          | Must pass or wrap objects; composition is ownership, not global lookup          |
| Composability (call unknown module that touches extra accounts) | Natural                                                                               | Awkward; extra objects must be in the tx                                        |

Sui's win is **not** “higher peak TPS in a lab block.” It is **removing owned traffic from consensus and from STM abort storms**. Aptos's win is **one ordered pipeline** that still parallelizes messy, account-shaped DeFi without asking the wallet for an object list.

## Consensus coupling

Aptos: every user tx is in a BFT block. Execution can pipeline after ordering (and Aptos

Aptos: every user tx is in a BFT block. Execution can pipeline after ordering (and Aptos block times are tens of ms), but you cannot finalize a coin transfer before the block that contains it. Block-STM only speeds *that* block.

Sui: ownership is a consensus-level type.

- Owned → fast path (Mysticeti-FPC / certified execution)
- Shared → Mysticeti-C assigns object versions, then execute

Execution and consensus share the object graph. Block-STM does not; it is an execution-layer engine you could bolt onto any ordered log (which is why Sei/Polygon/Starknet reused the idea).

## Move dialect

Same ancestry, different bytecode contract:

- Aptos: `move_to`, `borrow_global`, `signer`, sequence numbers / orderless nonces, multi-agent, tables, Objects as an add-on.
- Sui: no global ops; `UID + key`; `entry` takes objects; packages are objects; `init` on publish.

Safety story is shared (abilities, no implicit copy of assets). Scheduling story is not.

## Failure modes

|                       | Aptos                                                  | Sui                                                                         |
|-----------------------|--------------------------------------------------------|-----------------------------------------------------------------------------|
| Wrong access list     | N/A                                                    | Tx rejected or cannot touch the object                                      |
| Hidden dependency     | Re-execute until read set matches serial order         | Cannot happen if inputs are complete                                        |
| Hot object            | CPU burned on incarnations + validation waves          | Queue on that object’s versions; latency grows, CPU does not thrash as much |
| Opacity / dirty reads | VM aborts speculative inconsistency; scheduler retries | Owned path never speculates against a concurrent writer                     |

## For a game / metaverse economy

This is whv the earlier architecture leaned Sui-shaped. with STM as a fallback:

- Inventories, equipment, land deeds, character objects → **owned**. Sui-style fast path matches “I equipped a sword.”
- A single world auction house or raid boss HP → **shared**. Both chains serialize; Sui does it with object versions, Aptos with STM on one resource.
- A contract that walks “whatever is in this account” → Aptos is easier.
- A contract that is a tree of items → Sui is easier.

A hybrid (what NEMO-style research points at) is: **Sui objects for classification + Block-STM only on the shared-object subset**. Owned txs greedy-commit; shared txs get MVMemory and a scheduler.

## Which to copy

Choose **Block-STM** if you want:

- global storage / account model
- wallets that do not list keys
- an execution crate you can drop on any BFT log
- DeFi-shaped composability

Choose **Sui’s model** if you want:

- assets as first-class objects
- owned transfers that do not wait on the rest of the chain
- deterministic parallelism without abort storms
- game-shaped state

They are not two implementations of the same scheduler. Block-STM is an **optimistic engine over a total order**. Sui is a **causally scheduled object machine that only total-orders the shared subset**. For a Rust game L1, the data model choice matters more than which STM paper you cite.